Hibafelismerés és hibajavítás – gyakorlati feladatok megoldásai

Telekommunikációs hálózatok gyakorlat

Alapfogalmak, amiket a feladatokban használunk

- **Hamming-távolság** két kódszó között: azon bitpozíciók száma, ahol a két szó eltér.
- **A kód (minimális) Hamming-távolsága** *d*: bármely két érvényes kódszó közötti legkisebb távolság.
- A kód **d − 1** egyszerű bithibát biztosan felismer, és **⌊(d − 1)/2⌋** bithibát tud javítani.
- **Löketszerű (burst) hiba**: egymás melletti bitek egy szakasza, ahol a szakasz első és utolsó bitje biztosan hibás, a köztük lévők tetszőlegesen (a löket *hossza* a szakasz hossza).
- **Even parity**: a paritásbitet úgy választjuk, hogy az 1-esek száma (a paritásbittel együtt) páros legyen. Ez pont az, amit az 1. feladat kér: a paritásbit 1, ha az adatbitek közt páratlan sok 1-es van, különben 0.

1. feladat – egyetlen paritásbit

> *A 7 bites kódszavak alkotják a kódkönyvet. Adjunk hozzá egy paritásbitet, ami 1, ha a kódszóban lévő egyesek száma páratlan, és 0, ha páros.*

Példa

Kódszó: `1101011` – ebben 5 db 1-es van, ami páratlan, ezért a paritásbit 1:

```
adat:      1101011
paritás:          1
küldött:   1101011 1     (8 bit, az 1-esek száma így 6 = páros)
```

Ellenpélda: `1100011` – 4 db 1-es, páros → paritásbit 0 → küldött: `1100011 0`.

Hamming-távolság

d = 2. Ha két érvényes 8 bites szó csak egyetlen adatbitben különbözne, akkor az 1-esek paritása is eltérne, tehát a paritásbitjük is más lenne – így legalább 2 pozícióban különböznek. Két adatbitben eltérő szavak paritásbitje azonos, tehát a 2 el is érhető, pl. `1101011 1` és `1011011 1`.

Egyszerű és löketszerű hibák

- Felismer: **d − 1 = 1** egyszerű bithibát (bármely 1 bit, akár a paritásbit megváltozása is kiderül).
- Javít: **⌊(2 − 1)/2⌋ = 0** hibát – a vevő csak azt tudja, hogy baj van, azt nem, hol.
- Általánosan: minden **páratlan** számú bithibát felismer, **páros** számút soha (a paritás nem változik).
- Löketszerű hiba: csak az **1 hosszú** löketet ismeri fel garantáltan. Már egy 2 hosszú löket (két szomszédos bit megfordul) észrevétlen marad.

```
küldött:   1101011 1
vett:      1011011 1   ← a 2. és 3. bit megfordult, a paritás stimmel → NEM derül ki
vett:      1001011 0   ← egy adatbit ÉS a paritásbit fordult → NEM derül ki
vett:      1001011 1   ← csak a 2. bit fordult → 4 db 1-es, de a paritásbit 1 → hiba!
```

2. feladat – két paritásbit (páros és páratlan pozíciók)

Egyetlen paritásbit által nyújtottnál nagyobb biztonságot akarunk elérni, így olyan hibaészlelő sémát alkalmazunk, amelyben két paritásbit van: az egyik a páros, a másik a páratlan helyiértéken lévő bitek ellenőrzésére.

Példa

Even parityt használunk. Az adatbiteket 1-től számozzuk. A 8. pozíció páros, a 9. páratlan, ezért a páros pozíciókat ellenőrző bitet ($P_{páros}$) tesszük a 8., a páratlan pozíciókat ellenőrzőt ($P_{páratlan}$) a 9. helyre – így mindkét paritásbit a „saját” csoportjában marad.

```
pozíció:    1 2 3 4 5 6 7 | 8    9
adat:       1 0 1 0 1 1 0 | Ppáros Ppáratlan

páratlan pozíciók (1,3,5,7): 1 1 1 0 → 3 db 1-es → Ppáratlan = 1
páros pozíciók   (2,4,6):    0 0 1   → 1 db 1-es → Ppáros    = 1

küldött: 1010110 1 1
```

Érdemes úgy tekinteni rá, hogy ez **két egymásba fésült (interleaved) egyszerű paritáskód**: a páratlan pozíciójú bitek egy 4+1 bites, a párosak egy 3+1 bites paritásos kódszót alkotnak.

Hamming-távolság

d = 2. Egyetlen adatbit megváltoztatása pontosan az egyik csoport paritásbitjét fordítja meg, tehát bármely két érvényes kódszó legalább 2 bitben tér el; két azonos csoportba eső adatbit megváltoztatása (pl. 2. és 4. bit) pedig egyik paritásbitet sem változtatja, így a 2 el is érhető. A két külön paritásbit tehát **nem növeli** a Hamming-távolságot – a nyereség a löketek kezelésében van.

Egyszerű és löketszerű hibák

- Egyszerű hiba: **1** bithibát biztosan felismer, javítani **0**-t tud. Ráadásul a vevő azt is megtudja, melyik csoportban (páros vagy páratlan pozíción) történt a hiba – de hogy azon belül hol, azt nem.

```
Ppáratlan rossz, Ppáros jó  → páratlan pozíción van a hiba
Ppáros rossz, Ppáratlan jó  → páros pozíción van a hiba
mindkettő rossz            → mindkét csoportban (legalább) 1 hiba
```

- Általánosan: felismer minden olyan hibamintát, ahol **legalább az egyik csoportban páratlan** számú bit fordult meg. Nem ismeri fel, ha mindkét csoportban páros sok hiba van.
- Két tetszőleges hiba: ha ugyanabban a csoportban vannak, nem derül ki; ha külön csoportban, mindkét paritásbit jelez.

```
küldött: 1010110 1 1
vett:    1111110 1 1   ← 2. és 4. bit (mindkettő páros pozíció) → NEM derül ki
vett:    1110010 1 1   ← 2. és 5. bit (páros + páratlan) → mindkettő jelez
```

- Löketszerű hiba: egy löket egymás melletti biteket érint, azok pedig **felváltva** tartoznak a páros és páratlan csoportba. Egy legfeljebb **3 hosszú** löket az egyik csoportba legfeljebb 2, a másikba pontosan 1 bitet küld – az 1-et tartalmazó csoport paritásbitje biztosan jelez. Ezért ez a kód **minden 3 vagy rövidebb löketet felismer** (szemben az 1. feladat 1 hosszú löketével). 4 hosszú löket már 2–2 bitet érinthet, ami észrevétlen maradhat.

```
küldött: 1010110 1 1
vett:    1100110 1 1   ← 2 hosszú löket (2.,3. bit): mindkét paritás jelez
vett:    1101110 1 1   ← 3 hosszú löket (2–4. bit): 2 páros + 1 páratlan → Ppáratlan jelez
vett:    1101010 1 1   ← 4 hosszú löket (2–5. bit): 2 páros + 2 páratlan → NEM derül ki
```

3. feladat – mátrixos (oszlop-) paritás és kétdimenziós paritás

Tekintsük az n küldendő adatbitet, mint egy k×l bit-mátrixot. Minden oszlophoz számoljunk ki egy paritásbitet, és egészítsük ki a mátrixot egy új sorral, mely ezeket a paritásbiteket tartalmazza. Küldjük el az adatokat soronként. Példa k = 2, l = 3 esetén.

Even parityvel (ahogy a Moodle-példában) az oszlopparitások:

```
1 0 1
0 1 1
-----
1 1 0     ← oszloponként: 1 db, 1 db, 2 db 1-es → 1, 1, 0

küldött bitsorozat (soronként): 101 011 110
```

(Odd parityvel ugyanez `0 0 1` lenne – a paritás fajtája a továbbiakat nem befolyásolja.)

Egyszerű és löketszerű hibák

- Egyszerű bithiba: egyetlen bit megváltozása elrontja az érintett oszlop paritását → **felismerhető**. Az oszlop kiderül, a sor nem, ezért javítani nem lehet.
- Két hiba **ugyanabban az oszlopban**: kioltják egymást → **nem** ismerhető fel. Két hiba különböző oszlopokban: két oszlopparitás is jelez → felismerhető.
- Löketszerű hiba: mivel soronként küldünk, egy legfeljebb **l hosszú** löket minden oszlopot legfeljebb egyszer érint → minden ilyen löket felismerhető, bármilyen is a bitmintája. Ez a módszer fő előnye: a sima paritás 1 hosszú lökete helyett l hosszút kezel.

Milyen hosszú lehet egy csupa hibás bitből álló sorozat, hogy még felismerjük?

Ha a löket **minden** bitje megfordul, akkor egy hiba csak úgy maradhat észrevétlen, ha minden oszlopba páros számú (0 vagy 2) hiba esik. Egy l+1 … 2l−1 hosszú sorozat egy oszlopba ugyan kétszer is beleér, de lesz olyan oszlop, amelyet csak egyszer érint – az jelez. Pontosan 2l hosszú sorozat minden oszlopot kétszer fordít meg → észrevétlen. Tehát **legfeljebb 2l − 1** hosszú csupa-hibás sorozatot ismerünk fel biztosan.

```
l = 3, minden bit hibás:
hossz 3 (=l):    1 0 1 | 0 1 1 → 0 1 0 | 0 1 1   minden oszlop 1x → jelez
hossz 5 (=2l-1): 1 0 1 | 0 1 1 → 0 1 0 | 1 0 1   1.,2. oszlop 2x, 3. oszlop 1x → jelez
hossz 6 (=2l):   1 0 1 | 0 1 1 → 0 1 0 | 1 0 0   minden oszlop 2x → NEM derül ki
```

Összefoglalva: *tetszőleges* löketet **l** hosszig, *csupa hibás* löketet **2l − 1** hosszig ismer fel garantáltan.

Kétdimenziós paritás: új oszlop soronkénti paritásbitekkel

```
        sorparitás
1 0 1 | 0       ← 2 db 1-es → 0
0 1 1 | 0       ← 2 db 1-es → 0
------+--
1 1 0 | 0       ← a sarokbit a paritássor (1 1 0) paritása; ugyanez jön ki
                  a paritásoszlop (0 0) paritásaként is

küldött (soronként, sorparitással együtt): 1010 0110 1100
```

Hogyan javít 1 bithibát?

Egy bithiba pontosan egy sorparitást és egy oszlopparitást ront el. A rossz sor és a rossz oszlop **metszéspontja** megadja a hibás bit helyét, amit a vevő visszafordít.

```
vett:            paritásellenőrzés:
1 0 1 | 0        2 db 1-es, paritás 0 → OK
0 0 1 | 0        1 db 1-es, paritás 0 → HIBÁS sor
------+--
1 1 0 | 0        oszlopok: 1.: 1,0,1 → OK   2.: 0,0,1 → HIBÁS   3.: 1,1,0 → OK

→ hiba a 2. sor 2. bitjében: 0 → 1 visszajavítva:  0 1 1
```

Így a kód Hamming-távolsága 4: egy adatbit megváltoztatása a sor-, az oszlop- és a sarokparitást is megfordítja. d = 4 → 3 hibát felismer, 1-et javít.

Több bithiba és löketek

- 2 hiba **ugyanabban a sorban**: a sorparitás rendben marad, de 2 oszlopparitás jelez → felismerhető, de nem javítható (nem tudjuk, melyik sorban vannak). Ugyanez fordítva (egy oszlopban 2 hiba).
- 2 hiba **különböző sorban és oszlopban**: 2 sor- és 2 oszlopparitás jelez → felismerhető, de a hibák két „átlós” elhelyezkedése egyformán lehetséges, így nem javítható.
- 3 tetszőleges hiba: még mindig biztosan felismerhető (d = 4).
- 4 hiba egy **téglalap négy sarkában** (két sor × két oszlop): minden sor- és oszlopparitás rendben marad → **nem** ismerhető fel. Ez a legkisebb észrevétlen minta.

```
1 0 1 | 0        1 1 0 | 0
0 1 1 | 0   →    0 0 0 | 0     1. és 2. sor, 2. és 3. oszlop 4 bitje fordult meg,
1 1 0 | 0        1 1 0 | 0     minden paritás stimmel
```

- Löketek: soronként küldve egy legfeljebb egy sor hosszúságú (l+1 bit, a sorparitással együtt) löket minden oszlopot legfeljebb egyszer érint → felismerhető. Rövid löketek (a sor hosszánál rövidebbek) tehát biztosan kiderülnek; javítani csak akkor lehet, ha a löket egyetlen bitet ért.

4. feladat – CRC

Számolja ki a CRC kontrollösszeget az 1110.1101.0011.0100.0111 üzenetre, ha a generátorpolinom x^4+x^3+x+1! Adja meg a bitsorozatot, amit elküldünk!

Lépések

- A generátorpolinom bitalakja: x^4+x^3+x+1 → együtthatók (x^4, x^3, x^2, x, 1) → `G = 11011`, 5 bit → a CRC 4 bites lesz (fokszám = 4).
- Az üzenet végére 4 nullát fűzünk: `1110110100110100 0111 0000`.
- Modulo-2 osztás (XOR, nincs átvitel): mindig oda írjuk a generátort, ahol a pillanatnyi maradék 1-essel kezdődik. Az utolsó 4 bit a maradék.

```
11101101001101000111 0000
11011
-----
 11010100110100011100 0000
 11011
 -----
     11001101000111 0000
     11011
     -----
        1010100011 0000
        11011
        -----
         110000111 0000
         11011
         -----
           11001110 0000
           11011
           -----
             1111110 0000
             11011
             -----
               10010 0000
               11011
               -----
                1001 0000
                11011
                -----
                 100 0000
                 11011
                 -----
                  10 0000
                  11011
                  -----
                   1 0111
```

```
maradék (CRC) = 1001
```

A küldött bitsorozat az üzenet és a hozzáfűzött CRC:

```
1110 1101 0011 0100 0111 | 1001
111011010011010001111001   (24 bit)
```

Ellenőrzés: a küldött 24 bitet 11011-gyel osztva a maradék 0000.

Példa olyan bithibára, amit a CRC nem ismer fel

A vevő akkor nem vesz észre hibát, ha a vett sorozat is osztható G-vel, azaz ha a **hibaminta (a megfordított bitek) maga is G többszöröse**. A legegyszerűbb ilyen: magát a G-t

(11011) XOR-oljuk rá az üzenet egy tetszőleges helyére, azaz megfordítjuk a 11., 12., 14., 15. bitet:

```
küldött:    11101101001101000111001
hibaminta:  00000000001101100000000   (= G eltolva, G többszöröse)
vett:       11101101000000100111001

vett ÷ 11011 → maradék 0000 → a vevő hibátlannak hiszi
```

Ugyanígy észrevétlen minden olyan hibaminta, ami G bármely többszöröse (pl. G eltolt példányainak XOR-ja).

Történt-e hiba?

Történt-e hiba az átvitel során, ha a fogadó a következő üzenetet kapja: 1101.0011.0000.1010, és a generátorpolinom x^4+x^2+1? (A teljes fogadott üzenet, a kontrollösszeggel együtt.)

- `G = 10101` (x^4+x^2+1). A vevő a **teljes** vett sorozatot osztja G-vel (nullákat már nem fűz hozzá, mert a CRC benne van), és a maradékot nézi.

```
1101001100001010
10101
-----
 111101100001010
 10101
 -----
  10111100001010
  10101
  -----
     10100001010
     10101
     -----
         1001010
         10101
         -----
           11110
           10101
           -----
            1011

maradék = 1011 ≠ 0000
```

A maradék nem nulla, tehát **igen, hiba történt az átvitel során**. (Nulla maradék esetén sem lennénk teljesen biztosak a hibátlanságban – csak annyit tudnánk, hogy a hibaminta nem G többszöröse.)

Kiegészítő feladatok – keretezés (bit- és bájtbeszúrás)

Bitbeszúrás

Az 111111100110111 üzenet hogy fog kinézni a bitbeszúrásos módszer alkalmazása után?

Szabály: a küldő minden **öt egymást követő 1-es** után beszúr egy 0-t, függetlenül attól, hogy mi jön utána – így a 01111110 flag sosem fordulhat elő az adatban. A vevő öt 1-es után a 0-t eldobja.

```
eredeti:   11111 11 00110111
                ↑ öt 1-es után 0 beszúrása
küldött:   11111 0 11 00110111  =  1111101100110111   (16 bit)
```

A maradék részben (11 0011 0111) sehol sincs öt egymás utáni 1-es, így több beszúrás nem kell.

Bájtbeszúrás

A bájtbeszúrásos módszer esetén hogy kerül átvitelre a következő adat: A B C FLAG D ESC FLAG E F FLAG FLAG G

Szabály: az adatban előforduló minden FLAG és minden ESC bájt elé egy ESC-et szúrunk be. A vevő az ESC-et eldobja, és a rákövetkező bájtot adatként veszi.

```
adat:     A B C  FLAG   D  ESC      FLAG   E F  FLAG   FLAG   G
küldött:  A B C ESC FLAG D ESC ESC ESC FLAG E F ESC FLAG ESC FLAG G

(a keretet kívülről természetesen egy nyitó és egy záró FLAG fogja közre)
```

Eredeti adat hossza bájtbeszúrás után

Bájtalapú protokoll, a keretek FLAG bájttal kezdődnek és végződnek, bájtbeszúrást használ. 8 keretet küldünk, a keretek összhossza a médiumon 2048 bájt. Az eredeti adatban pontosan 16 ESC és 8 FLAG bájt fordult elő. Hány bájtból állt az eredeti adat?

```
keretezés:   8 keret × 2 FLAG (nyitó + záró)          = 16 bájt
beszúrás:    16 adatbeli ESC  → mindegyik elé +1 ESC   = 16 bájt
             8 adatbeli FLAG  → mindegyik elé +1 ESC   =  8 bájt
-------------------------------------------------------------
többlet összesen:                                       40 bájt

eredeti adat = 2048 − 40 = 2008 bájt
```